

Natural Language-Driven Robotic Manipulation System with Integrated Computer Vision and LLM Control

Jatin Satyam

School for Engineering of Matter, Transport and Energy
Ira A. Fulton Schools of Engineering, Arizona State University, Tempe, AZ
Email: jsatyam@asu.edu

Abstract—This work introduces an intelligent robotic manipulation framework that leverages natural language processing through Large Language Models (LLMs) combined with computer vision for autonomous object handling. The implemented system utilizes a Dobot Magician Lite robotic platform integrated with sophisticated AI-driven control mechanisms for interpreting conversational instructions, identifying objects through RGB camera input, and performing precise pick-and-place operations. The framework encompasses visual scene analysis, HSV-based color detection, affine transformation for coordinate mapping, intelligent state management with memory persistence, and stack-aware block manipulation. Experimental validation demonstrates successful execution of complex multi-step operations commanded through natural language, confirming the viability of combining linguistic reasoning with physical robotic control systems.

Index Terms—Natural Language Processing, Robotic Manipulation, Computer Vision, Large Language Models, Autonomous Systems, Object Detection, Coordinate Transformation, Dobot Magician.

I. INTRODUCTION

The integration of natural language understanding into robotic systems represents a significant advancement in human-robot interaction paradigms. Conventional robotic platforms typically demand explicit programming with rigid command structures. However, recent developments in Large Language Model technology have opened new possibilities for intuitive, conversation-based robot control.

This research presents a comprehensive autonomous manipulation system capable of processing and executing natural language directives, including:

- “Retrieve the nearest red cube and position it adjacent to the green object.”
- “Take the farthest blue block and construct a stack with the yellow piece.”
- “Transfer all red blocks to the designated placement zone.”

The architecture combines linguistic interpretation via LLMs, vision-based object recognition, calibrated coordinate transformation, and kinematically validated robotic motion control.

II. SYSTEM ARCHITECTURE

A. Hardware Configuration

The experimental platform comprises:

- Dobot Magician Lite articulated robotic manipulator
- AI-compatible USB RGB camera system
- Standardized colored manipulation blocks
- Computing workstation running Python 3.10 with OpenCV and LLM integration



Fig. 1. Experimental setup shows execution of the maze-solving program and the Dobot Magician Lite positioned on top of the maze board.

B. Software Framework

The system architecture consists of modular components:

- **robot_manipulation_system.py** — Central orchestration module with LLM integration and state management
- **vision_block_locator.py** — Camera interface and HSV-based object detection
- **robot_m.py** — Low-level motion primitives and kinematic control
- **open_camera.py** — Camera initialization and frame acquisition utilities

III. IMPLEMENTATION METHODOLOGY

A. Natural Language Command Processing

The LLM subsystem performs:

- 1) Parsing and semantic analysis of user instructions
- 2) Object identification and target determination
- 3) Action classification (stacking, positioning, rotation)
- 4) Code generation for robotic execution

The system maintains conversational context through persistent state tracking, enabling multi-turn interactions and reference resolution (e.g., “that block,” “the stack”).

B. Visual Perception and Object Detection

Object localization employs HSV color space segmentation with morphological filtering to extract:

- Object identifiers with per-color indexing (red_1, blue_2, etc.)
- Pixel-space centroids (x, y coordinates)
- Contour area measurements for size estimation
- Real-time detection with JSON serialization

The detection algorithm applies adaptive thresholding to accommodate varying illumination conditions, with configurable HSV ranges for each target color.

C. Coordinate System Transformation

Pixel coordinates are mapped to robot workspace coordinates using an affine transformation matrix:

$$\begin{bmatrix} X_{robot} \\ Y_{robot} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & b_1 \\ a_{21} & a_{22} & b_2 \end{bmatrix} \cdot \begin{bmatrix} u_{pixel} \\ v_{pixel} \\ 1 \end{bmatrix}$$

Calibration employs least-squares regression over multiple reference point correspondences, achieving sub-millimeter accuracy across the operational workspace.

D. State Management and Memory Persistence

The system implements intelligent state tracking featuring:

- Real-time block position monitoring
- Stack height detection and maintenance
- Gripper status awareness
- Persistent state storage across sessions
- Automatic state recovery after system restart

Stack detection uses spatial clustering with configurable tolerance to group blocks at identical locations, tracking height and composition dynamically.

E. Motion Planning and Execution

The manipulation pipeline executes:

- 1) Safe approach trajectory to source position
- 2) Controlled descent with height-aware z-offset
- 3) Vacuum gripper activation for secure grasp
- 4) Clearance elevation to safe travel height
- 5) Trajectory generation to target location
- 6) Precision placement with stack-aware positioning
- 7) Gripper release and state update

All motions incorporate safety checks, workspace boundary validation, and collision avoidance for stacked configurations.

IV. EXPERIMENTAL RESULTS

The system successfully executed diverse natural language commands, including:

- Single-block retrieval and placement
- Multi-block stacking operations
- Directional positioning (“next to,” “beside,” “on top of”)
- Color-based selection (“nearest red,” “farthest blue”)
- Sequential multi-step operations

A. Performance Metrics

- Command interpretation accuracy: High reliability across varied phrasing
- Object detection success rate: Consistent under controlled lighting
- Positioning precision: $\pm 2\text{mm}$ in xy-plane
- Execution time: 15-30 seconds per pick-and-place cycle

B. Video Documentation

Demonstration Part 1: https://youtube.com/shorts/_HYQI9ThReQ?feature=share

V. ANALYSIS AND DISCUSSION

The integration of LLM reasoning with computer vision and robotic control enables intuitive task specification without explicit programming. The persistent state management proved essential for maintaining context across multi-step operations.

Key observations include:

- LLM code generation provides flexibility but requires validation
- Color segmentation performance depends critically on lighting consistency
- Affine transformation calibration significantly impacts positioning accuracy
- Stack-aware manipulation requires careful z-height management

Challenges encountered included ambient lighting variations affecting color detection and the need for periodic recalibration of the transformation matrix.

VI. CONCLUSIONS AND FUTURE WORK

This research demonstrates a functional autonomous manipulation system combining natural language understanding, computer vision, and robotic control. The modular architecture supports extensibility for additional sensors, more complex manipulation primitives, and enhanced reasoning capabilities.

Future enhancements could include:

- Integration of depth sensing for improved 3D positioning
- Expanded object recognition beyond color-based detection
- Multi-robot coordination through shared state management
- Advanced grasp planning with force feedback

The system establishes a foundation for scalable, language-driven robotic automation in real-world scenarios.

ACKNOWLEDGMENT

The author would like to thank **Prof. Sangram Redkar** for his guidance throughout the RAS 545 course and laboratory sessions. Gratitude is also extended to **Sai Srinivas Tatwik Meesala** and **Rajesh S. Aouti** for their assistance during the experiment and for providing valuable insights into Dobot operation, calibration, and safety.

REFERENCES

- [1] RAS 545 Laboratory Manual, "Experiments with Dobot Magician Lite Platform," Arizona State University, 2025.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] LangChain Development Team, "LangChain: Building Applications with LLMs," 2024.
- [4] Groq Inc., "Groq LPU Inference Engine Documentation," 2024.
- [5] L. Samsoniuk, "PyDobot: Python Interface for Dobot Robotic Arms," GitHub Repository, 2023.

APPENDIX A

ROBOT_MANIPULATION_SYSTEM.PY

Main orchestration module implementing LLM integration, state management, and high-level control logic.

```
#!/usr/bin/env python3
"""
INTELLIGENT ROBOTIC MANIPULATION SYSTEM - LLM
Enhanced

Advanced automation framework featuring:
    Multi-object color-based tracking and manipulation
    Stack-aware positioning with persistent memory
    Natural language command processing via LLM
    Robust coordinate transformation (camera robot space)
    Safe execution with validation and error handling

System Components:
    BlockDetector: Computer vision for object identification
    DobotController: Precision robotic arm control
    SystemState: Intelligent state tracking with persistence
```

```
LLMCodeGenerator: Natural language interpretation
CodeExecutor: Validated command execution
PickPlaceOrchestrator: High-level system orchestration

"""
from __future__ import annotations

import argparse
import ast
import json
import os
import sys
import time
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import cv2 # type: ignore
import numpy as np # type: ignore
from langchain_groq import ChatGroq # type: ignore
from langchain_core.messages import HumanMessage, SystemMessage # type: ignore

__version__ = "1.0.0"

#
# System Configuration Parameters
#

LLM_API_KEY = os.environ.get("LLM_API_KEY", None)
if not LLM_API_KEY:
    print("[WARN] LLM_API_KEY is not set. Natural-language control will be disabled.")
os.environ["LLM_API_KEY"] = LLM_API_KEY or ""

ROBOT_PORT = os.environ.get("ROBOT_PORT", "/dev/ttyACM0")
CAMERA_INDEX = int(os.environ.get("CAMERA_INDEX", "0"))

# Camera frame (used for bounds clamping when computing adjacent placements)
FRAME_WIDTH = 640
FRAME_HEIGHT = 480

# State file for persistent memory
PERSISTENCE_FILE = Path(os.environ.get("SYSTEM_PERSISTENCE_FILE", str(Path.home() / "dobot_state.json"))))

# Dobot Positions
ROBOT_HOME_POS = {'x': 250.0, 'y': 0.0, 'z': 150.0, 'r': 0.0}

# Z-heights (you may need to tweak these for your table/blocks)
GRAB_ELEVATION = -48.0
CLEARANCE_HEIGHT = 50.0
DROP_ELEVATION = -44.0
CUBE_HEIGHT = 17.9 # mm vertical spacing between stacked blocks

# In-plane block spacing
CUBE_DIMENSION_MM = 15.0
ADJACENT_SPACING_MM = CUBE_DIMENSION_MM
```

```

75 # Optional fine-tuning offsets in mm (for
76 camera robot misalignment)
77 PICK_X_OFFSET_MM = float(os.environ.get("
78 PICK_X_OFFSET_MM", "0"))
79 PICK_Y_OFFSET_MM = float(os.environ.get("
80 PICK_Y_OFFSET_MM", "0"))
81 PICK_Z_OFFSET_MM = float(os.environ.get("
82 PICK_Z_OFFSET_MM", "0"))
83 PLACE_X_OFFSET_MM = float(os.environ.get("
84 PLACE_X_OFFSET_MM", "0"))
85 PLACE_Y_OFFSET_MM = float(os.environ.get("
86 PLACE_Y_OFFSET_MM", "0"))
87 PLACE_Z_OFFSET_MM = float(os.environ.get("
88 PLACE_Z_OFFSET_MM", "0"))
89
90 # Calibration points (pixel dobot XY)
91 #COORD_REFERENCE_POINTS = [
92 #{'pixel': (583, 451), 'dobot':
93 # (214.077,-110.869)},
94 #{'pixel': (33, 450), 'dobot': (214.889,140.137)},
95 #{'pixel': (302, 309), 'dobot': (283.350,17.730)},
96 #{'pixel': (40, 195), 'dobot': (335.087,137.740)},
97 #{'pixel': (33, 450), 'dobot':
98 # (347.299,-112.861)},
99 #]
100 COORD_REFERENCE_POINTS = [
101 {'pixel': (318, 244), 'dobot':
102 (274.9905090332031, 13.742196083068848)},
103 {'pixel': (183, 351), 'dobot':
104 (224.83358764648438, 72.8918685913086)},
105 {'pixel': (458, 365), 'dobot':
106 (222.3900146484375, -51.34326934814453)}, #
107 # fixed
108 {'pixel': (193, 125), 'dobot': (327.1572265625,
109 66.56614685058594)},
110 {'pixel': (468, 131), 'dobot':
111 (328.6787414550781, -58.42145919799805)},
112 ]
113 #
114
115 # Coordinate Transformation Module
116 #
117
118 class CoordinateConverter:
119     """Converts pixel coordinates to Dobot
120     coordinates using affine transformation."""
121
122     def __init__(self) -> None:
123         if len(COORD_REFERENCE_POINTS) < 3:
124             raise ValueError("Need at least 3
125             calibration points for affine
126             transform.")
127         self.calibration_points =
128             COORD_REFERENCE_POINTS
129         self.avg_r = 14.25 # default wrist rotation
130         self.calculate_transformation()
131
132         # Derived mm-per-pixel scale estimates (for
133         "next to" spacing etc.)
134         self.mm_per_px_x = float(np.sqrt(self.a11 **2
135         + self.a21 ** 2))
136         self.mm_per_px_y = float(np.sqrt(self.a12 **2
137         + self.a22 ** 2))
138         print(f"[CoordSys] Calibration loaded (
139             mm/px: x {self.mm_per_px_x:.3f}, y {
140
141             self.mm_per_px_y:.3f})")
142
143     def calculate_transformation(self) -> None:
144         """Calculate affine transformation matrix
145         from calibration points."""
146         pixel_coords = np.array([
147             [p['pixel'][0], p['pixel'][1], 1] for p
148             in self.calibration_points
149         ])
150         dobot_x = np.array([p['dobot'][0] for p in
151             self.calibration_points])
152         dobot_y = np.array([p['dobot'][1] for p in
153             self.calibration_points])
154
155         coeffs_x, _, _, _ = np.linalg.lstsq(
156             pixel_coords, dobot_x, rcond=None)
157         self.a11, self.a12, self.b1 = coeffs_x
158
159         coeffs_y, _, _, _ = np.linalg.lstsq(
160             pixel_coords, dobot_y, rcond=None)
161         self.a21, self.a22, self.b2 = coeffs_y
162
163         print("[CoordSys] Transformation
164             coefficients calculated")
165
166     def pixel_to_dobot(self, pixel_x: int, pixel_y:
167         int) -> Tuple[float, float]:
168         """Convert pixel coordinates to Dobot XY
169         coordinates."""
170         dobot_x = self.a11 * pixel_x + self.a12 *
171             pixel_y + self.b1
172         dobot_y = self.a21 * pixel_x + self.a22 *
173             pixel_y + self.b2
174         print(f"[Transform] Pixel ({pixel_x}, {
175             pixel_y}) Dobot ({dobot_x:.1f}, {
176             dobot_y:.1f})")
177         return float(dobot_x), float(dobot_y)
178
179     def mm_to_pixels(self, dx_mm: float = 0.0, dy_mm
180         : float = 0.0, axis: str = 'x') -> int:
181         """Convert desired delta in mm to pixels
182         along x or y axis."""
183         if axis == 'x':
184             if self.mm_per_px_x <= 1e-6:
185                 return 0
186             return int(round(dx_mm / self.
187                 mm_per_px_x))
188         else:
189             if self.mm_per_px_y <= 1e-6:
190                 return 0
191             return int(round(dy_mm / self.
192                 mm_per_px_y))
193
194 #
195
196 # State Management and Persistence
197 #
198
199 class SystemState:
200     """
201     State tracking with persistence and stack
202     awareness.
203
204     For each block we remember:
205     - color
206     - per-color id (1,2,3,...)
207     - pixel position (x,y)
208     - whether it's on the table / in a stack / in
209       the gripper
210     - which blocks are below it (for stacks)

```

```

169 """
170
171 def __init__(self, state_file: Path =
172     PERSISTENCE_FILE, coord_converter: Optional[
173     CoordinateConverter] = None) -> None:
174     self.state_file = state_file
175     self.gripper_status: str = "EMPTY"
176     self.holding_block: Optional[str] = None
177     self.blocks: Dict[str, Dict[str, Any]] = {}
178     self.conversation_history: List[Dict[str,
179     Any]] = []
180     self.stack_map: Dict[str, List[str]] = {} #
181     {location_key: [block_ids bottomtop
182     ]}
183     self.stack_meta: Dict[str, Dict[str, Any]] =
184     {} # {location_key: {'x':px, 'y':px,
185     'height':int, 'updated_at':iso}}
186     self.last_place_location: Optional[Dict[str,
187     Any]] = None
188     self.last_manipulated_block_id: Optional[str]
189     = None # "that"/"it" resolver
190     self.min_adjacent_clearance_mm: float =
191     ADJACENT_SPACING_MM
192
193     self.coord_converter: CoordinateConverter =
194     coord_converter or CoordinateConverter()
195
196 # ----- stack helper internals
197 -----
198
199 def _location_key(self, pixel_x: int, pixel_y:
200     int, tolerance: int = 30) -> str:
201     """Create location key for grouping blocks
202     at same position (bucketing by ~
203     tolerance px)."""
204     x_bucket = round(pixel_x / tolerance) *
205     tolerance
206     y_bucket = round(pixel_y / tolerance) *
207     tolerance
208     return f"{x_bucket},{y_bucket}"
209
210 def _update_stack_meta_for_key(self, loc_key:
211     str) -> None:
212     """Update centroid (avg pixel) and height
213     for a stack location key."""
214     if loc_key not in self.stack_map or len(self
215     .stack_map[loc_key]) == 0:
216         if loc_key in self.stack_meta:
217             del self.stack_meta[loc_key]
218         return
219
220     bids = self.stack_map[loc_key]
221     xs = [self.blocks[bid]['pixel_x'] for bid in
222     bids if bid in self.blocks]
223     ys = [self.blocks[bid]['pixel_y'] for bid in
224     bids if bid in self.blocks]
225     if not xs or not ys:
226         return
227
228     cx = int(round(sum(xs) / len(xs)))
229     cy = int(round(sum(ys) / len(ys)))
230     self.stack_meta[loc_key] = {
231         'x': cx,
232         'y': cy,
233         'height': len(bids),
234         'updated_at': datetime.now().isoformat()}
235
236 def rebuild_stacks(self) -> None:
237     """
238     Rebuild stack_map and stack_meta based
239     purely on block positions.
240
241     Only blocks that are NOT in the gripper are
242
243     considered.
244     """
245     self.stack_map = {}
246     self.stack_meta = {}
247
248     for block_id, block in self.blocks.items():
249         if block.get('in_gripper'):
250             continue
251         loc_key = self._location_key(block['
252         pixel_x'], block['pixel_y'])
253         self.stack_map.setdefault(loc_key, []).
254         append(block_id)
255
256     # Sort each stack by known stack_level (if
257     available)
258     for loc_key, ids in self.stack_map.items():
259         self.stack_map[loc_key] = sorted(
260             ids,
261             key=lambda bid: self.blocks.get(bid,
262             {}).get('stack_level', 0)
263         )
264         self._update_stack_meta_for_key(loc_key)
265
266 # ----- block initialization /
267 updates -----
268
269 def initialize_blocks(self, detected_blocks:
270     List[Dict[str, Any]]) -> None:
271     """Initialize blocks from camera detection."""
272     self.blocks = {}
273     self.stack_map = {}
274     self.stack_meta = {}
275
276     for block in detected_blocks:
277         block_id = block['global_id']
278         self.blocks[block_id] = {
279             'color': block['color'],
280             'id': block['id'],
281             'pixel_x': block['pixel_x'],
282             'pixel_y': block['pixel_y'],
283             'original_x': block['pixel_x'],
284             'original_y': block['pixel_y'],
285             'current_position': {'x': block['
286             pixel_x'], 'y': block['pixel_y']
287             }, 'z': 0.0,
288             'stack_level': 0, # 0 = on table
289             'blocks_below': [],
290             'in_gripper': False
291         }
292
293     self.rebuild_stacks()
294     print(f"[StateTracker] Initialized {len(self
295     .blocks)} blocks")
296     self.save_state()
297
298 def update_gripper(self, block_id: Optional[str]
299     = None) -> None:
300     """
301     Update gripper status + in_gripper flags +
302     stack membership.
303
304     Convention:
305     - When called with a block_id: you have just
306     picked that block.
307       mark it in_gripper=True and remove it
308       from any stack it belonged to.
309     - When called with None: you just released
310     the currently held block.
311       its in_gripper flag will be cleared
312       when you call update_block_position
313       for the place location; here we just
314       mark gripper as empty.
315     """

```

```

276     if block_id is None:
277         # Releasing whatever we were holding
278         print("[StateTracker] Gripper: now EMPTY")
279         self.gripper_status = "EMPTY"
280         # We DO NOT clear in_gripper here;
281         # update_block_position will handle it
282         self.holding_block = None
283     else:
284         # Picking this block
285         self.gripper_status = "HOLDING"
286         self.holding_block = block_id
287         self.last_manipulated_block_id = block_id
288
289         block = self.blocks.get(block_id)
290         if block:
291             block['in_gripper'] = True
292             loc_key = self._location_key(block['pixel_x'], block['pixel_y'])
293             if loc_key in self.stack_map and block_id in self.stack_map[loc_key]:
294                 self.stack_map[loc_key].remove(block_id)
295                 if not self.stack_map[loc_key]:
296                     del self.stack_map[loc_key]
297                     if loc_key in self.stack_meta:
298                         del self.stack_meta[loc_key]
299                     else:
300                         self._update_stack_meta_for_key(loc_key)
301
302             print(f"[StateTracker] Gripper status: {self.gripper_status}, holding: {self.holding_block}")
303             self.save_state()
304
305     def update_block_position(self, block_id: str, pixel_x: int, pixel_y: int, z: float = 0.0, blocks_below: Optional[List[str]] = None) -> None:
306         """
307         Update block position with stack awareness.
308
309         This is called after placing a block. It:
310         - sets new pixel & z,
311         - adds it to the stack at that location,
312         - records which blocks are below,
313         - marks in_gripper=False.
314
315         """
316         if block_id not in self.blocks:
317             print(f"[StateTracker] update_block_position called on unknown block_id={block_id}")
318             return
319
320         block = self.blocks[block_id]
321
322         # Remove from old stack location if present
323         old_loc_key = self._location_key(block['pixel_x'], block['pixel_y'])
324         if old_loc_key in self.stack_map and block_id in self.stack_map[old_loc_key]:
325             self.stack_map[old_loc_key].remove(block_id)
326             if not self.stack_map[old_loc_key]:
327                 del self.stack_map[old_loc_key]
328                 if old_loc_key in self.stack_meta:
329                     del self.stack_meta[old_loc_key]
330                 else:
331                     self._update_stack_meta_for_key(old_loc_key)
332
333         del self.stack_map[old_loc_key]
334         if old_loc_key in self.stack_meta:
335             del self.stack_meta[old_loc_key]
336         else:
337             self._update_stack_meta_for_key(old_loc_key)
338
339         # Update position
340         block['pixel_x'] = pixel_x
341         block['pixel_y'] = pixel_y
342         block['current_position'] = {'x': pixel_x, 'y': pixel_y, 'z': z}
343         block['in_gripper'] = False
344
345         # Calculate stack level from z offset (0 if on table)
346         stack_level = round(z / CUBE_HEIGHT) if z > 0 else 0
347         block['stack_level'] = stack_level
348
349         # Update blocks_below list
350         block['blocks_below'] = (blocks_below or [])
351
352         # Add to new stack location
353         new_loc_key = self._location_key(pixel_x, pixel_y)
354         self.stack_map.setdefault(new_loc_key, [])
355         if block_id not in self.stack_map[new_loc_key]:
356             self.stack_map[new_loc_key].append(block_id)
357
358         # Sort by z level and refresh meta
359         self.stack_map[new_loc_key].sort(key=lambda bid: self.blocks[bid]['stack_level'])
360
361         self._update_stack_meta_for_key(new_loc_key)
362
363         # Track last place location + last manipulated
364         self.last_place_location = {'x': pixel_x, 'y': pixel_y, 'z': z}
365         self.last_manipulated_block_id = block_id
366
367         print(f"[StateTracker] {block_id} moved to ({pixel_x}, {pixel_y}, z={z}, level={stack_level})")
368         self.save_state()
369
370     # ----- state queries -----
371
372     def get_stack_at_location(self, pixel_x: int, pixel_y: int) -> List[str]:
373         """Get stack of blocks at a location (list bottomtop)."""
374         loc_key = self._location_key(pixel_x, pixel_y)
375         return self.stack_map.get(loc_key, [])
376
377     def get_stack_height(self, pixel_x: int, pixel_y: int) -> int:
378         """Get number of blocks stacked at location. """
379         return len(self.get_stack_at_location(pixel_x, pixel_y))
380
381     def get_block(self, block_id: str) -> Optional[Dict[str, Any]]:
382         """Get block data by ID."""
383         return self.blocks.get(block_id)
384
385     def is_gripper_empty(self) -> bool:

```

```

386         return self.gripper_status == "EMPTY"
387
388     def get_holding_block_id(self) -> Optional[str]:
389         return self.holding_block
390
391     def get_last_manipulated_block_id(self) ->
392         Optional[str]:
393         """Use for pronouns like 'that' or 'it'
394             referring to the previously handled
395             block."""
396         return self.last_manipulated_block_id
397
398     def _distance_to_robot_home_mm(self, pixel_x:
399         int, pixel_y: int) -> float:
400         """Euclidean distance in mm from
401             ROBOT_HOME_POS in Dobot XY space."""
402         dx, dy = self.coord_converter.pixel_to_dobot
403             (pixel_x, pixel_y)
404         return float(np.hypot(dx - ROBOT_HOME_POS['x'],
405             dy - ROBOT_HOME_POS['y']))
406
407     def _nearest_or_farthest_block(self, block_ids:
408         List[str], mode: str = 'nearest') ->
409         Optional[str]:
410         # Ignore blocks currently in the gripper
411         block_ids = [bid for bid in block_ids if not
412             self.blocks.get(bid, {}).get('
413                 in_gripper')]
414         if not block_ids:
415             return None
416         scored: List[Tuple[float, str]] = []
417         for bid in block_ids:
418             b = self.blocks.get(bid)
419             if not b:
420                 continue
421             d = self._distance_to_robot_home_mm(b['
422                 pixel_x'], b['pixel_y'])
423             scored.append((d, bid))
424         if not scored:
425             return None
426         scored.sort(key=lambda t: t[0])
427         return scored[0][1] if mode == 'nearest'
428         else scored[-1][1]
429
430     def get_nearest_block_id_by_color(self, color:
431         str) -> Optional[str]:
432         ids = [bid for bid, b in self.blocks.items()
433             if b['color'].lower() == color.lower
434             () and not b.get('in_gripper')]
435         return self._nearest_or_farthest_block(ids,
436             mode='nearest')
437
438     def get_farthest_block_id_by_color(self, color:
439         str) -> Optional[str]:
440         ids = [bid for bid, b in self.blocks.items()
441             if b['color'].lower() == color.lower
442             () and not b.get('in_gripper')]
443         return self._nearest_or_farthest_block(ids,
444             mode='farthest')
445
446     def get_nearest_block_id(self) -> Optional[str]:
447         ids = [bid for bid, b in self.blocks.items()
448             if not b.get('in_gripper')]
449         return self._nearest_or_farthest_block(ids,
450             mode='nearest')
451
452     def get_farthest_block_id(self) -> Optional[str]:
453         ids = [bid for bid, b in self.blocks.items()
454             if not b.get('in_gripper')]
455         return self._nearest_or_farthest_block(ids,
456             mode='farthest')
457
458     def get_stack_locations(self, min_height: int =
459         2) -> List[Dict[str, Any]]:
460         """
461         Return list of dicts for each stack with
462         height >= min_height:
463         [{'loc_key': str, 'x': int, 'y': int, '
464             height': int}, ...], tallest first.
465         """
466         out: List[Dict[str, Any]] = []
467         for loc_key, meta in self.stack_meta.items():
468             h = int(meta.get('height', 0))
469             if h >= min_height:
470                 out.append({'loc_key': loc_key, 'x':
471                     int(meta['x']), 'y': int(meta['
472                         y']), 'height': h})
473         out.sort(key=lambda d: d['height'], reverse=
474             True)
475         return out
476
477     def get_tallest_stack_xy(self, min_height: int =
478         2) -> Optional[Tuple[int, int, int]]:
479         """Return (x, y, height) for tallest stack (
480             height>=2). None if no such stack."""
481         stacks = self.get_stack_locations(min_height
482             =min_height)
483         if not stacks:
484             return None
485         s = stacks[0]
486         return int(s['x']), int(s['y']), int(s['
487             height'])
488
489     def get_nearest_stack_xy(self, min_height: int =
490         2) -> Optional[Tuple[int, int, int]]:
491         """Return (x, y, height) for nearest stack
492             to robot. None if no stack."""
493         stacks = self.get_stack_locations(min_height
494             =min_height)
495         if not stacks:
496             return None
497         best = None
498         bestd = None
499         for s in stacks:
500             d = self._distance_to_robot_home_mm(int(
501                 s['x']), int(s['y']))
502             if best is None or d < bestd: # type:
503                 ignore
504                 best = s
505                 bestd = d
506         assert best is not None
507         return int(best['x']), int(best['y']), int(
508             best['height'])
509
510     # ----- "next to" helpers
511     -----
512
513     def compute_adjacent_position(
514         self,
515         ref_x: int,
516         ref_y: int,
517         direction: str = 'right',
518         clearance_mm: Optional[float] = None
519     ) -> Tuple[int, int]:
520         """
521         Compute a pixel position adjacent to (ref_x,
522             ref_y) with at least clearance_mm
523             center-to-center.
524         Directions: 'right' (+x), 'left' (-x), '
525             front' (-y, toward top), 'back' (+y,
526             toward bottom).
527         Uses calibrated mm-per-pixel to produce safe
528             offsets. Coordinates are clamped to
529             frame bounds.
530         """
531         base_clearance = self.

```

```

486         min_adjacent_clearance_mm if
487         clearance_mm is None else float(
488         clearance_mm)
489     clr = base_clearance + 3.0 # 3mm safety
490     margin
491
492     dx_px = 0
493     dy_px = 0
494
495     if direction == 'right':
496         dx_px = self.coord_converter.
497             mm_to_pixels(dx_mm=clr, axis='x')
498     elif direction == 'left':
499         dx_px = -self.coord_converter.
500             mm_to_pixels(dx_mm=clr, axis='x')
501     elif direction == 'front':
502         dy_px = -self.coord_converter.
503             mm_to_pixels(dy_mm=clr, axis='y')
504     elif direction == 'back':
505         dy_px = self.coord_converter.
506             mm_to_pixels(dy_mm=clr, axis='y')
507     else:
508         dx_px = self.coord_converter.
509             mm_to_pixels(dx_mm=clr, axis='x')
510
511     nx = int(np.clip(ref_x + dx_px, 0,
512                     FRAME_WIDTH - 1))
513     ny = int(np.clip(ref_y + dy_px, 0,
514                     FRAME_HEIGHT - 1))
515     print(f"[StateTracker] Adjacent ({direction}
516           }, {clr}mm): ({ref_x}, {ref_y}) ({
517           nx}, {ny}) [dx_px={dx_px}, dy_px={dy_px}]
518           ")
519     return nx, ny
520
521 def compute_adjacent_to_block(
522     self,
523     block_id: str,
524     direction: str = 'right',
525     clearance_mm: Optional[float] = None
526 ) -> Optional[Tuple[int, int]]:
527     """Adjacent pixel (x,y) next to the given
528     block id."""
529     b = self.blocks.get(block_id)
530     if not b:
531         print(f"[StateTracker]
532               compute_adjacent_to_block: unknown
533               block_id={block_id}")
534         return None
535     if b.get('in_gripper'):
536         print(f"[StateTracker]
537               compute_adjacent_to_block: {block_id
538               } is in gripper; using last known
539               table position")
540     return self.compute_adjacent_position(b['
541     pixel_x'], b['pixel_y'], direction=
542     direction, clearance_mm=clearance_mm)
543
544 def compute_adjacent_to_stack(
545     self,
546     direction: str = 'right',
547     strategy: str = 'nearest',
548     clearance_mm: Optional[float] = None
549 ) -> Optional[Tuple[int, int]]:
550     """
551     Adjacent pixel XY next to a chosen stack (
552     2 high).
553     strategy:
554         - 'nearest' : stack closest to robot
555         - 'tallest' : highest stack
556     """
557     if strategy == 'tallest':
558         sel = self.get_tallest_stack_xy(
559             min_height=2)
560
561     else:
562         sel = self.get_nearest_stack_xy(
563             min_height=2)
564
565     if not sel:
566         print("[StateTracker]
567               compute_adjacent_to_stack: no stacks
568               (height>=2) available")
569         return None
570     sx, sy, _ = sel
571     return self.compute_adjacent_position(sx, sy
572     , direction=direction, clearance_mm=
573     clearance_mm)
574
575 # ----- conversation + persistence
576 -----
577
578 def add_to_history(self, prompt: str, action:
579 str, details: Optional[Dict[str, Any]] =
580 None) -> None:
581     """Add command to history with optional
582     details."""
583     entry = {
584         'prompt': prompt,
585         'action': action,
586         'timestamp': datetime.now().isoformat(),
587         'details': details or {}
588     }
589     self.conversation_history.append(entry)
590     if len(self.conversation_history) > 20:
591         self.conversation_history = self.
592             conversation_history[-20:]
593     self.save_state()
594
595 def get_state_summary(self) -> str:
596     """Get formatted state for LLM with enhanced
597     context."""
598     summary = f"""CURRENT SYSTEM STATE:
599 - Gripper Status: {self.gripper_status}
600 - Holding Block: {self.holding_block or 'None'}
601 - Last Manipulated Block: {self.
602     last_manipulated_block_id or 'None'}
603
604 AVAILABLE BLOCKS (with stacking info):
605 """
606     for block_id, block in self.blocks.items():
607         stack_info = ""
608         if block.get('in_gripper'):
609             stack_info = " [IN GRIPPER]"
610         elif block['stack_level'] > 0:
611             stack_info = f" [STACKED - Level {
612                 block['stack_level']}, on: {'
613                 ',
614                 '.join(block['blocks_below'])}]"
615         summary += f" - {block_id}: {block['
616         color']} at pixel ({block['pixel_x
617         ']}, {block['pixel_y']}){stack_info
618         }\n"
619
620 # Show stacks
621 if self.stack_map:
622     summary += "\nDETECTED STACKS:\n"
623     for loc_key, stack_blocks in self.
624         stack_map.items():
625         if len(stack_blocks) > 1:
626             meta = self.stack_meta.get(
627                 loc_key, {})
628             cx = meta.get('x', '?')
629             cy = meta.get('y', '?')
630             stack_descr = " ".join(
631                 f"{bid} ({self.blocks[bid]['
632                 color']})" for bid in
633                 stack_blocks if bid in
634                 self.blocks
635             )
636             summary += f" - Location {

```

```

        loc_key} @ ({cx},{cy}): {
            stack_descr} ({len(
                stack_blocks)} high)\n"

# Show last place location
if self.last_place_location:
    summary += f"\nLAST PLACE LOCATION:
        pixel ({self.last_place_location['x'
        ']}), {self.last_place_location['y'
        ']}), z={self.last_place_location['z
        ']}]\n"

# Show recent actions
if self.conversation_history:
    summary += f"\nRECENT ACTIONS (last {min
        (5, len(self.conversation_history))
        }):\n"
    for entry in self.conversation_history
        [-5:]:
        details_str = ""
        if entry.get('details'):
            details_str = f" | {entry['
            details']}"
        summary += f" - '{entry['prompt']}'
            {entry['action']}{
            details_str}\n"

    return summary

def save_state(self) -> None:
    """Save state to JSON file."""
    try:
        state_data = {
            'gripper_status': self.
                gripper_status,
            'holding_block': self.holding_block,
            'blocks': self.blocks,
            'conversation_history': self.
                conversation_history,
            'stack_map': self.stack_map,
            'stack_meta': self.stack_meta,
            'last_place_location': self.
                last_place_location,
            'last_manipulated_block_id': self.
                last_manipulated_block_id,
            'timestamp': datetime.now().
                isoformat()
        }
        with open(self.state_file, 'w') as f:
            json.dump(state_data, f, indent=2)
        print(f"[StateTracker] Saved to {
            self.state_file}")
    except Exception as e:
        print(f"[StateTracker] Save
            failed: {e}")

def load_state(self) -> bool:
    """Load state from JSON file."""
    try:
        if not self.state_file.exists():
            print(f"[StateTracker] No saved
                state found at {self.state_file
                }")
            return False
        with open(self.state_file, 'r') as f:
            state_data = json.load(f)
        self.gripper_status = state_data.get('
            gripper_status', 'EMPTY')
        self.holding_block = state_data.get('
            holding_block')
        self.blocks = state_data.get('blocks',
            {})
        self.conversation_history = state_data.
            get('conversation_history', [])

```

```

        self.stack_map = state_data.get('
            stack_map', {})
        self.stack_meta = state_data.get('
            stack_meta', {})
        self.last_place_location = state_data.
            get('last_place_location')
        self.last_manipulated_block_id =
            state_data.get('
            last_manipulated_block_id')
        print(f"[StateTracker] Loaded state
            from {self.state_file}")
        print(f"[StateTracker] Restored: {len(
            self.blocks)} blocks, {len(self.
            conversation_history)} history
            entries")

# Ensure newer field exists
for block in self.blocks.values():
    block.setdefault('in_gripper', False
        )
    block.setdefault('stack_level', 0)
    block.setdefault('blocks_below', [])
    block.setdefault('current_position',
        {
            'x': block.get('pixel_x', 0),
            'y': block.get('pixel_y', 0),
            'z': 0.0
        })

# Rebuild stacks to be safe
self.rebuild_stacks()

return True
except Exception as e:
    print(f"[StateTracker] Load
        failed: {e}")
    return False

#
# -----
# Dobot controller
# -----

class DobotController:
    """Hardware control for Dobot Magician Lite."""

    def __init__(self, port: str = ROBOT_PORT,
        coord_converter: Optional[
            CoordinateConverter] = None) -> None:
        self.port = port
        self.device = None
        self.coord_converter: CoordinateConverter =
            coord_converter or CoordinateConverter()

    def connect(self, port: Optional[str] = None) ->
        bool:
        """Connect to Dobot."""
        port = port or self.port
        try:
            from pydobot import Dobot # type:
                ignore
        except Exception as e:
            print(f"[RobotCtrl] ERROR: pydobot
                not available ({e})")
            return False

        try:
            print(f"\n[RobotCtrl] Connecting to {
                port}...")
            self.device = Dobot(port=port)

```

```

687         time.sleep(1)
688         if hasattr(self.device, 'set_speed'):
689             self.device.set_speed(velocity=150,
690                                   acceleration=150)
691         print("[RobotCtrl] Connected successfully!")
692         return True
693     except Exception as e:
694         print(f"[RobotCtrl] ERROR: {e}")
695         self.device = None
696         return False
697
698 def move_to(self, x: float, y: float, z: float,
699             r: float = 0.0, wait: bool = True) -> None:
700     """Move Dobot to coordinates."""
701     if self.device:
702         try:
703             # Try calling with wait parameter
704             first
705             self.device.move_to(x, y, z, r, wait=
706                                 wait)
707         except TypeError:
708             # If wait parameter not supported,
709             # call without it
710             self.device.move_to(x, y, z, r)
711             if wait:
712                 time.sleep(0.2)
713
714 def go_home(self) -> None:
715     """Move to home position."""
716     print("[RobotCtrl] Moving to home position
717           ...")
718     self.move_to(ROBOT_HOME_POS['x'],
719                  ROBOT_HOME_POS['y'], ROBOT_HOME_POS['z']
720                  ], ROBOT_HOME_POS['r'])
721     print("[RobotCtrl] At home position")
722
723 def suction_on(self) -> None:
724     """Enable suction cup."""
725     if self.device:
726         self.device.suck(True)
727         print("[RobotCtrl] Suction: ON")
728     time.sleep(0.8)
729
730 def suction_off(self) -> None:
731     """Disable suction cup."""
732     if self.device:
733         self.device.suck(False)
734         print("[RobotCtrl] Suction: OFF")
735     time.sleep(0.5)
736
737 def pick_block(self, pixel_x: int, pixel_y: int,
738                z_offset: float = 0.0) -> None:
739     """Pick up block at pixel coordinates, with
740       optional z_offset for stacked blocks."""
741     print(f"\n[RobotCtrl] Picking block at pixel
742           ({pixel_x}, {pixel_y}) (z_offset={
743             z_offset:.1f}mm)...")
744     dobot_x, dobot_y = self.coord_converter.
745         pixel_to_dobot(pixel_x, pixel_y)
746     dobot_x += PICK_X_OFFSET_MM
747     dobot_y += PICK_Y_OFFSET_MM
748     target_z = GRAB_ELEVATION + PICK_Z_OFFSET_MM
749     + z_offset
750     print(f"[RobotCtrl] Pick pose: x={dobot_x:.1
751           f}, y={dobot_y:.1f}, z={target_z:.1f}")
752     self.move_to(dobot_x, dobot_y,
753                  CLEARANCE_HEIGHT)
754     self.move_to(dobot_x, dobot_y, target_z)
755     self.suction_on()
756     self.move_to(dobot_x, dobot_y,
757                  CLEARANCE_HEIGHT)
758     print("[RobotCtrl] Block picked")
759
760 def place_block(self, pixel_x: int, pixel_y: int,
761                 z_offset: float = 0.0) -> None:
762     """Place block at pixel coordinates with
763       optional Z offset for stacking."""
764     print(f"\n[RobotCtrl] Placing block at pixel
765           ({pixel_x}, {pixel_y}), z_offset={
766             z_offset:.1f}mm...")
767     dobot_x, dobot_y = self.coord_converter.
768         pixel_to_dobot(pixel_x, pixel_y)
769     dobot_x += PLACE_X_OFFSET_MM
770     dobot_y += PLACE_Y_OFFSET_MM
771     place_z = DROP_ELEVATION + PLACE_Z_OFFSET_MM
772     + z_offset
773     print(f"[RobotCtrl] Place pose: x={dobot_x
774           :.1f}, y={dobot_y:.1f}, z={place_z:.1f}")
775     self.move_to(dobot_x, dobot_y,
776                  CLEARANCE_HEIGHT)
777     self.move_to(dobot_x, dobot_y, place_z)
778     self.suction_off()
779     self.move_to(dobot_x, dobot_y,
780                  CLEARANCE_HEIGHT)
781     print("[RobotCtrl] Block placed")
782
783 def close(self) -> None:
784     """Close Dobot connection."""
785     if self.device:
786         try:
787             self.device.close()
788             print("[RobotCtrl] Connection closed
789                   ")
790         except Exception:
791             pass
792         finally:
793             self.device = None
794
795 # -----
796 # Block detection
797 # -----
798
799 class BlockDetector:
800     """Camera-based block detection."""
801
802     def __init__(self, camera_index: int =
803                  CAMERA_INDEX) -> None:
804         self.camera_index = camera_index
805         self.camera = None
806         # HSV ranges; tweak per camera if needed
807         self.color_ranges = {
808             'red': [
809                 (np.array([0, 100, 100]), np.array
810                  ([10, 255, 255])),
811                 (np.array([160, 100, 100]), np.array
812                  ([180, 255, 255]))
813             ],
814             'blue': [(np.array([103, 140, 60]), np.
815                       array([130, 255, 255]))],
816             'green': [(np.array([40, 100, 100]), np.
817                       array([80, 255, 255]))],
818             'yellow': [(np.array([20, 100, 100]), np.
819                        array([35, 255, 255]))]
820         }
821
822     def detect_color_blocks(self, frame: np.ndarray,
823                             color_name: str) -> List[Dict[str, Any]]:
824         """Detect blocks of a specific color in a
825           frame."""
826         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
827         mask = None

```

```

794     for lower, upper in self.color_ranges[
795         color_name]:
796         if mask is None:
797             mask = cv2.inRange(hsv, lower, upper
798                 )
799         else:
800             mask = cv2.bitwise_or(mask, cv2.
801                 inRange(hsv, lower, upper))
802             kernel = np.ones((5, 5), np.uint8)
803             mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,
804                 kernel)
805             mask = cv2.morphologyEx(mask, cv2.
806                 MORPH_CLOSE, kernel)
807             contours, _ = cv2.findContours(mask, cv2.
808                 RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
809             detected: List[Dict[str, Any]] = []
810             for contour in contours:
811                 area = cv2.contourArea(contour)
812                 if 200 < area < 10000:
813                     M = cv2.moments(contour)
814                     if M["m00"] != 0:
815                         cx = int(M["m10"] / M["m00"])
816                         cy = int(M["m01"] / M["m00"])
817                         detected.append({'center': (cx,
818                             cy), 'contour': contour, '
819                             area': area})
820             return detected
821
822 def detect_all_blocks_live(self) -> Optional[
823     List[Dict[str, Any]]]:
824     """Open camera and detect all blocks in real
825     -time (manual spacebar capture)."""
826     print(f"\n[Camera] Opening camera {self.
827         camera_index}...")
828     try:
829         self.camera = cv2.VideoCapture(self.
830             camera_index)
831         if not self.camera.isOpened():
832             print("[Camera] ERROR: Failed to
833                 open camera")
834             return None
835             print("[Camera] Camera opened
836                 successfully")
837     except Exception as e:
838         print(f"[Camera] ERROR: {e}")
839         return None
840
841     self.camera.set(cv2.CAP_PROP_FRAME_WIDTH,
842         FRAME_WIDTH)
843     self.camera.set(cv2.CAP_PROP_FRAME_HEIGHT,
844         FRAME_HEIGHT)
845
846     print("[Camera] Press SPACE to capture | Q
847         to quit")
848     detected_blocks: Optional[List[Dict[str, Any]
849         ]] = []
850
851     while True:
852         ret, frame = self.camera.read()
853         if not ret:
854             break
855
856         display = frame.copy()
857         all_detected: List[Dict[str, Any]] = []
858         for color_name in ['red', 'blue', 'green',
859             'yellow']:
860             blocks = self.detect_color_blocks(
861                 frame, color_name)
862             for block in blocks:
863                 all_detected.append({'color':
864                     color_name, 'center': block[
865                         'center'], 'contour': block[
866                             'contour']})
867
868         # Sorting is purely for consistent
869         # labeling
870         all_detected.sort(key=lambda b: (b['
871             center'][1] // 50, b['center'][0]))
872
873         vis_colors = {'red': (0, 0, 255), 'blue'
874             : (255, 0, 0), 'green': (0, 255, 0),
875             'yellow': (0, 255, 255)}
876         temp_blocks: List[Dict[str, Any]] = []
877         temp_color_counters = {'red': 1, 'blue':
878             1, 'green': 1, 'yellow': 1}
879
880         for block in all_detected:
881             cx, cy = block['center']
882             color_name = block['color']
883             color_bgr = vis_colors[color_name]
884             color_id = temp_color_counters[
885                 color_name]
886             temp_color_counters[color_name] += 1
887
888             cv2.drawContours(display, [block['
889                 contour']], -1, color_bgr, 2)
890             cv2.circle(display, (cx, cy), 5,
891                 color_bgr, -1)
892
893             label = f"{color_name}_{color_id}"
894             cv2.putText(display, label, (cx -
895                 30, cy - 20), cv2.
896                 FONT_HERSHEY_SIMPLEX, 0.6, (255,
897                 255, 255), 2)
898             cv2.putText(display, f"({cx},{cy})",
899                 (cx - 30, cy + 30), cv2.
900                 FONT_HERSHEY_SIMPLEX, 0.4, (255,
901                 255, 255), 1)
902
903             temp_blocks.append({'color':
904                 color_name, 'id': color_id, '
905                 center': (cx, cy)})
906
907         cv2.putText(display, f"Blocks: {len(
908             all_detected})", (10, 30),
909             cv2.FONT_HERSHEY_SIMPLEX,
910             0.7, (0, 255, 0), 2)
911         cv2.imshow("Block Detection", display)
912
913         key = cv2.waitKey(1) & 0xFF
914         if key == ord(' '):
915             if temp_blocks:
916                 detected_blocks = []
917                 for block in temp_blocks:
918                     global_id = f"{block['color']
919                         }_{block['id']}"
920                     detected_blocks.append({'
921                         global_id': global_id,
922                         'color': block['color'],
923                         'id': block['id'],
924                         'pixel_x': block['center']
925                             [0],
926                         'pixel_y': block['center']
927                             [1]
928                     })
929                 print(f"\n[Camera] Captured
930                     {len(detected_blocks)}
931                     blocks")
932                 break
933             elif key == ord('q'):
934                 print("[Camera] Detection cancelled")
935                 detected_blocks = None
936                 break
937
938         self.camera.release()
939         cv2.destroyAllWindows()
940         return detected_blocks

```

```

895
896 def auto_capture_blocks(self, display_duration:
      float = 2.0) -> Optional[List[Dict[str, Any
      ]]]:
897     """Auto-capture blocks after showing live
      feed for a short duration."""
898     print("\n[Camera] Auto-capturing block
      positions...")
899     try:
900         camera = cv2.VideoCapture(self.
      camera_index)
901         if not camera.isOpened():
902             print("[Camera] ERROR: Failed to
      open camera")
903             return None
904         camera.set(cv2.CAP_PROP_FRAME_WIDTH,
      FRAME_WIDTH)
905         camera.set(cv2.CAP_PROP_FRAME_HEIGHT,
      FRAME_HEIGHT)
906
907         vis_colors = {'red': (0, 0, 255), 'blue'
      : (255, 0, 0), 'green': (0, 255, 0), 'yellow'
      : (0, 255, 255)}
908         start_time = time.time()
909         latest_blocks: List[Dict[str, Any]] = []
910
911         while True:
912             ret, frame = camera.read()
913             if not ret:
914                 break
915             display = frame.copy()
916             all_detected: List[Dict[str, Any]] = []
917             for color_name in ['red', 'blue', '
      green', 'yellow']:
918                 blocks = self.
      detect_color_blocks(frame,
      color_name)
919                 for block in blocks:
920                     all_detected.append({'color'
      : color_name, 'center':
      block['center'], '
      contour': block['contour']})
921             all_detected.sort(key=lambda b: (b['
      center'][1] // 50, b['center'
      ][0]))
922
923             temp_color_counters = {'red': 1, '
      blue': 1, 'green': 1, 'yellow': 1}
924             temp_blocks: List[Dict[str, Any]] = []
925             for block in all_detected:
926                 cx, cy = block['center']
927                 color_name = block['color']
928                 color_bgr = vis_colors[
      color_name]
929                 color_id = temp_color_counters[
      color_name]
930                 temp_color_counters[color_name]
      += 1
931
932                 cv2.drawContours(display, [block[
      'contour']], -1, color_bgr,
      2)
933                 cv2.circle(display, (cx, cy), 5,
      color_bgr, -1)
934
935                 label = f"{color_name}_{color_id}"
936                 cv2.putText(display, label, (cx
      - 30, cy - 20),
      cv2.
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

FONT_HERSHEY_SIMPLEX
, 0.6, (255,
255, 255), 2)
temp_blocks.append({
    'global_id': label,
    'color': color_name,
    'id': color_id,
    'pixel_x': cx,
    'pixel_y': cy
})

latest_blocks = temp_blocks
elapsed = time.time() - start_time
cv2.putText(display, f"AUTO-CAPTURE
- Blocks: {len(all_detected)}",
(10, 30),
cv2.FONT_HERSHEY_SIMPLEX
, 0.7, (0, 255, 0),
2)
cv2.imshow("Auto Block Capture",
display)
cv2.waitKey(30)
if elapsed >= display_duration:
    break

camera.release()
cv2.destroyAllWindows()
time.sleep(0.2)
return latest_blocks
except Exception as e:
    print(f"[Camera] ERROR: {e}")
    return None

#
# -----
# LLM code generator
# -----

class LLMCodeGenerator:
    """LLM wrapper that turns natural language into
    safe Python code for this system."""

    def __init__(self, api_key: Optional[str]) ->
    None:
        self.enabled = bool(api_key)
        self.llm: Optional[ChatGroq] = None

        if self.enabled:
            self.llm = ChatGroq(
                temperature=0.1,
                model_name="llama-3.3-70b-versatile"
                ,
                api_key=api_key or ""
            )
        else:
            print("[NLP] Disabled (no LLM_API_KEY);
            natural-language commands will not
            work.")

        self.system_prompt_template = """You are a
        contextually-aware Python code generator
        for a Dobot robotic arm.

        {state_summary}

        ALLOWED FUNCTIONS:
        - dobot.pick_block(pixel_x, pixel_y, z_offset=0)
        - dobot.place_block(pixel_x, pixel_y, z_offset=0)
        - system_state.update_gripper(block_id or None)
        - system_state.update_block_position(block_id,

```

993	- system_state.get_block(block_id)	1041	- This works for blocks on the table (level = 0)
994	- system_state.get_stack_at_location(pixel_x,	1042	and stacked blocks (level > 0).
	pixel_y)	1043	
995	- system_state.get_stack_height(pixel_x, pixel_y)	1044	4. Placing blocks:
996	- system_state.is_gripper_empty()	1045	- Before placing at (x, y), compute:
997	- system_state.get_holding_block_id()		stack_height = system_state.get_stack_height(
998	- system_state.get_last_manipulated_block_id()	1046	x, y)
999	- system_state.get_nearest_block_id_by_color(	1047	z_offset = stack_height * 17.9
	color_name)		blocks_below = system_state.
1000	- system_state.get_farthest_block_id_by_color(	1048	get_stack_at_location(x, y)
	color_name)	1049	- Then:
1001	- system_state.get_nearest_block_id()	1050	dobot.place_block(x, y, z_offset)
1002	- system_state.get_farthest_block_id()		system_state.update_block_position(block_id,
1003	- system_state.get_tallest_stack_xy()		x, y, z_offset, blocks_below=blocks_below
1004	- system_state.get_nearest_stack_xy()	1051	)
1005	- system_state.compute_adjacent_position(x, y,	1052	system_state.update_gripper(None)
	direction='right'/'left'/'front'/'back',	1053	
	clearance_mm=15.0)	1054	5. "Next to" placement:
1006	- system_state.compute_adjacent_to_block(block_id,		- When asked to place a block "next to" another
	direction='right'/'left'/'front'/'back',	1055	block:
	clearance_mm=15.0)		nx, ny = system_state.
1007	- system_state.compute_adjacent_to_stack(direction='		compute_adjacent_to_block(target_id,
	right'/'left'/'front'/'back', strategy='nearest'	1056	direction="right", clearance_mm=15.0)
	'/'tallest', clearance_mm=15.0)		- When asked to place a block "next to the stack"
1008	- time.sleep(seconds)	1057	:
1009			- If "nearest stack" is implied (just "the
1010	RULES:	1058	stack"):
1011	- OUTPUT ONLY PYTHON CODE. No explanations, no		nx, ny = system_state.
	comments, no markdown fences.		compute_adjacent_to_stack(direction="
1012	- Use ONLY the allowed functions and Python builtins		right", strategy="nearest",
	(print, len, range, int, float, str, round).	1059	clearance_mm=15.0)
1013		1060	- If "tallest stack" is specified:
1014	SEMANTICS & INTERPRETATION RULES:		nx, ny = system_state.
1015			compute_adjacent_to_stack(direction="
1016	1. Multi-block colors:		right", strategy="tallest",
1017	- There can be multiple blocks of each color (e.g.	1061	clearance_mm=15.0)
	. 2 red, 3 yellow, 2 green, 1 blue).	1062	- Then place using logic from rule 4.
1018	- For phrases like "the red block" or "a red	1063	
	block", choose a specific one using:		6. Color-to-color stacking (e.g. "place the farthest
1019	* nearest: system_state.	1064	red block on the nearest green block"):
	get_nearest_block_id_by_color("red")	1065	- Example pattern:
1020	* farthest: system_state.		red_id = system_state.
	get_farthest_block_id_by_color("red")	1066	get_farthest_block_id_by_color("red")
1021	- Phrases like "the farthest red block" use		green_id = system_state.
	get_farthest_block_id_by_color("red").	1067	get_nearest_block_id_by_color("green")
1022	- Phrases like "the nearest green block" use	1068	red = system_state.get_block(red_id)
	get_nearest_block_id_by_color("green").	1069	green = system_state.get_block(green_id)
1023		1070	# pick red
1024	2. Stacks:	1071	pick_z = red['stack_level'] * 17.9
1025	- A *stack* is a location with height >= 2 blocks	1072	dobot.pick_block(red['pixel_x'], red['pixel_y
	.	1073	'], z_offset=pick_z)
1026	- "the stack" (without qualifiers) the	1074	system_state.update_gripper(red_id)
	NEAREST stack:		# place on top of stack where green lives
1027	(x, y, h) = system_state.get_nearest_stack_xy()	1075	target_x = green['pixel_x']
		1076	target_y = green['pixel_y']
1028	- "the tallest stack" or "the highest stack"		h = system_state.get_stack_height(target_x,
	use:	1077	target_y)
1029	(x, y, h) = system_state.get_tallest_stack_xy()	1078	z = h * 17.9
		1079	dobot.place_block(target_x, target_y, z)
1030	- "over the stack", "on top of the stack"		below = system_state.get_stack_at_location(
	place on TOP of that chosen stack:	1080	target_x, target_y)
1031	z_offset = h * 17.9		system_state.update_block_position(red_id,
1032			target_x, target_y, z, blocks_below=below
1033	3. Picking blocks (including stacked ones):	1081	)
1034	- ALWAYS call system_state.update_gripper(	1082	system_state.update_gripper(None)
	block_id) after dobot.pick_block(...).	1083	
1035	- Use stack_level to pick at the correct height:	1084	7. Contextual references:
1036	block = system_state.get_block(block_id)		- "there" or "same place" use system_state.
1037	level = block['stack_level']	1085	last_place_location (x,y,z).
1038	pick_z = level * 17.9		- "that", "it", or "the previous block" use
1039	dobot.pick_block(block['pixel_x'], block['	1086	system_state.get_last_manipulated_block_id()).
	pixel_y'], z_offset=pick_z)	1087	
1040	system_state.update_gripper(block_id)	1088	8. Closest/Farthest (no color):
			- "nearest block" system_state.

1089	get_nearest_block_id()	1142	
	- "farthest block" system_state.	1143	CODE:
	get_farthest_block_id()	1144	yellow_id = system_state.
1090			get_nearest_block_id_by_color("yellow")
1091	9. Smart Return Handling:	1145	yellow = system_state.get_block(yellow_id)
1092	- If holding a different block and asked to pick	1146	pick_z = yellow['stack_level'] * 17.9
	a new one, FIRST place the held block back at	1147	dobot.pick_block(yellow['pixel_x'], yellow['pixel_y
	its current position		'], z_offset=pick_z)
1093	(using its current pixel_x, pixel_y and stack	1148	system_state.update_gripper(yellow_id)
	height), then pick the new one.	1149	time.sleep(0.3)
1094		1150	sx, sy, h = system_state.get_tallest_stack_xy()
1095	10. IMPORTANT EXECUTION STYLE:	1151	z = h * 17.9
1096	- This environment is stateful across commands.	1152	dobot.place_block(sx, sy, z)
	Always rely on:	1153	below = system_state.get_stack_at_location(sx, sy)
1097	* stack_level, in_gripper, stack_map,	1154	system_state.update_block_position(yellow_id, sx, sy
	last_place_location, etc.,		, z, blocks_below=below)
1098	instead of hardcoding assumptions.	1155	system_state.update_gripper(None)
1099		1156	
1100	EXAMPLE Command: "pick the farthest red block	1157	CRITICAL: Always use stack_level when picking, and
	and place it on the nearest green block"		stack_height when placing.
1101		1158	
1102	CODE:	1159	Generate executable Python code for:
1103	red_id = system_state.get_farthest_block_id_by_color	1160	"""
	("red")	1161	
1104	green_id = system_state.	1162	def generate_code(self, user_prompt: str,
	get_nearest_block_id_by_color("green")		system_state: SystemState) -> Optional[str]:
1105	red = system_state.get_block(red_id)	1163	"""Generate Python code from natural
1106	green = system_state.get_block(green_id)		language."""
1107	# pick farthest red	1164	if not self.enabled or self.llm is None:
1108	pick_z = red['stack_level'] * 17.9	1165	print("[NLP] Not available (missing API
1109	dobot.pick_block(red['pixel_x'], red['pixel_y'],		key).")
	z_offset=pick_z)	1166	return None
1110	system_state.update_gripper(red_id)	1167	
1111	time.sleep(0.3)	1168	state_summary = system_state.
1112	# place on stack at green location		get_state_summary()
1113	target_x = green['pixel_x']	1169	system_prompt = self.system_prompt_template.
1114	target_y = green['pixel_y']		format(state_summary=state_summary)
1115	h = system_state.get_stack_height(target_x, target_y	1170	messages = [
	)	1171	SystemMessage(content=system_prompt),
1116	z = h * 17.9	1172	HumanMessage(content=user_prompt)
1117	dobot.place_block(target_x, target_y, z)	1173	
1118	below = system_state.get_stack_at_location(target_x,	1174	try:
	target_y)	1175	print(f"\n[NLP] Analyzing: '{user_prompt
1119	system_state.update_block_position(red_id, target_x,		}')")
	target_y, z, blocks_below=below)	1176	print(f"[NLP] Current gripper state: {
1120	system_state.update_gripper(None)		system_state.gripper_status}")
1121		1177	response = self.llm.invoke(messages)
1122	EXAMPLE Command: "place the blue block right	1178	generated_code = response.content.strip
	next to the stack"		()
1123	(choose nearest stack, put blue on table next to it)	1179	
1124		1180	# Strip markdown fences if the model
1125	CODE:		ignores instructions
1126	blue_id = system_state.get_nearest_block_id_by_color	1181	if """python" in generated_code:
	("blue")	1182	generated_code = generated_code.
1127	blue = system_state.get_block(blue_id)		split("""python")[1].split("""
1128	pick_z = blue['stack_level'] * 17.9		")[0].strip()
1129	dobot.pick_block(blue['pixel_x'], blue['pixel_y'],	1183	elif """ in generated_code:
	z_offset=pick_z)	1184	generated_code = generated_code.
1130	system_state.update_gripper(blue_id)		split("""")[1].split("""")[0].
1131	time.sleep(0.3)		strip()
1132	nx, ny = system_state.compute_adjacent_to_stack(	1185	
	direction="right", strategy="nearest",	1186	print(f"[NLP] Code generated ({len(
	clearance_mm=15.0)		generated_code}) chars)")
1133	h = system_state.get_stack_height(nx, ny)	1187	return generated_code
1134	z = h * 17.9	1188	except Exception as e:
1135	dobot.place_block(nx, ny, z)	1189	print(f"[NLP] ERROR: {e}")
1136	below = system_state.get_stack_at_location(nx, ny)	1190	import traceback
1137	system_state.update_block_position(blue_id, nx, ny,	1191	traceback.print_exc()
	z, blocks_below=below)	1192	return None
1138	system_state.update_gripper(None)	1193	
1139		1194	
1140	EXAMPLE Command: "place the yellow block over	1195	#
	the stack"		
1141	(assume tallest stack)		

1196	# Code validator & executor	1251	)")
1197	#	1252	return False
	-----	1253	
1198		1254	try:
1199	class CodeExecutor:	1255	for node in ast.walk(tree):
1200	"""Validates and safely executes LLM-generated code."""	1256	# Block imports
1201		1257	if isinstance(node, (ast.Import, ast
1202	def __init__(self) -> None:	1258	.ImportFrom)):
1203	# Full dotted names for attribute calls	1259	print("[Validator] Imports
1204	self.allowed_calls = {	1260	are not allowed")
1205	'dobot.pick_block',	1261	return False
1206	'dobot.place_block',	1262	
1207	'system_state.update_gripper',	1263	# Block user-defined functions,
1208	'system_state.update_block_position',	1264	classes, lambdas
1209	'system_state.get_block',	1265	if isinstance(node, (ast.FunctionDef
1210	'system_state.get_stack_at_location',	1266	, ast.AsyncFunctionDef, ast.
1211	'system_state.get_stack_height',	1267	ClassDef, ast.Lambda)):
1212	'system_state.is_gripper_empty',	1268	print("[Validator] Defining
1213	'system_state.get_holding_block_id',	1269	functions/classes/lambdas is
1214	'system_state.	1270	not allowed")
1215	get_last_manipulated_block_id',	1271	return False
1216	'system_state.	1272	
1217	get_nearest_block_id_by_color',	1273	# Block while loops (safety)
1218	'system_state.	1274	if isinstance(node, ast.While):
1219	get_farthest_block_id_by_color',	1275	print("[Validator] while-
1220	'system_state.get_nearest_block_id',	1276	loops are not allowed")
1221	'system_state.get_tallest_stack_xy',	1277	return False
1222	'system_state.get_nearest_stack_xy',	1278	
1223	'system_state.compute_adjacent_position',	1279	if isinstance(node, ast.Call):
1224	'system_state.compute_adjacent_to_block',	1280	name = self._get_call_name(node)
1225	'system_state.compute_adjacent_to_stack',	1281	if name is None:
1226	'time.sleep',	1282	print("[Validator]
1227	} # Bare name calls allowed (Python builtins we expose in __builtins__)	1283	Blocked complex call (
1228	self.allowed_bare_calls = {'print', 'len', 'range', 'int', 'float', 'str', 'round'}	1284	not a simple attribute
1229	self.max_code_length = 4000	1285	or name)")
1230		1286	return False
1231	def _get_call_name(self, node: ast.Call) -> Optional[str]:	1287	
1232	"""Return dotted name for a call if it's an attribute on a simple Name, else None."""	1288	# Attribute call like
1233		1289	system_state.get_block
1234	func = node.func	1290	if '.' in name:
1235	if isinstance(func, ast.Attribute) and isinstance(func.value, ast.Name):	1291	if name not in self.
1236	base = func.value.id	1292	allowed_calls:
1237	attr = func.attr	1293	print(f"[Validator]
1238	return f"{base}.{attr}"	1294	Blocked call: {name}
1239	elif isinstance(func, ast.Name):	1295	return False
1240	return func.id # bare name call	1296	else:
1241	return None	1297	# Bare-name call: must be in
1242		1298	allowed_bare_calls
1243	def validate_code(self, code_string: str) -> bool:	1299	if name not in self.
1244	"""Validate code against whitelist."""	1300	allowed_bare_calls:
1245	if len(code_string) > self.max_code_length:	1301	print(f"[Validator]
1246	print(f"[Validator] Code too long ({len(code_string)} chars > {self.max_code_length})")	1302	Blocked bare call: {
1247	return False	1303	name}")
1248		1304	return False
1249	try:	1305	
1250	tree = ast.parse(code_string, mode='exec')	1306	print("[Validator] Code validated")
		1307	return True
		1308	except Exception as e:
		1309	print(f"[Validator] Validation error
		1310	: {e}")
		1311	return False
		1312	
		1313	def execute_code(self, code_string: str, dobot:
		1314	DobotController, system_state: SystemState)
		1315	-> bool:
		1316	"""Execute validated code in controlled
		1317	environment."""
		1318	if not self.validate_code(code_string):
		1319	print("[Executor] Code validation failed
		1320	, NOT executing")
		1321	return False
		1322	
		1323	safe_globals = {

```

1300         'dobot': dobot,
1301         'system_state': system_state,
1302         'time': time,
1303         '__builtins__': {
1304             'print': print,
1305             'len': len,
1306             'range': range,
1307             'int': int,
1308             'float': float,
1309             'str': str,
1310             'round': round,
1311         }
1312     }
1313     try:
1314         print("\n[Executor] Executing code...")
1315         print("-" * 70)
1316         exec(code_string, safe_globals)
1317         print("-" * 70)
1318         print("[Executor]      Execution completed")
1319         return True
1320     except Exception as e:
1321         print(f"\n[Executor]      EXECUTION ERROR")
1322         : {e}")
1323         import traceback
1324         traceback.print_exc()
1325         return False
1326
1327 #
1328 # Orchestrator
1329 #
1330
1331 class PickPlaceOrchestrator:
1332     """Main system coordinator with persistent
1333        memory."""
1334
1335     def __init__(self, port: str = ROBOT_PORT,
1336                  camera_index: int = CAMERA_INDEX) -> None:
1337         coord_converter = CoordinateConverter()
1338         self.dobot = DobotController(port=port,
1339                                     coord_converter=coord_converter)
1340         self.detector = BlockDetector(camera_index=
1341                                     camera_index)
1342         self.system_state = SystemState(
1343             coord_converter=coord_converter)
1344         self.llm = LLMCodeGenerator(LLM_API_KEY)
1345         self.executor = CodeExecutor()
1346
1347     def auto_update_blocks(self) -> None:
1348         """Auto-capture and update block positions
1349            XY only, keep known stack heights."""
1350         print("\n" + "=" * 70)
1351         print("      AUTO-UPDATING BLOCK POSITIONS..")
1352         print("=" * 70)
1353
1354         self.dobot.go_home()
1355         time.sleep(0.5)
1356
1357         captured_blocks = self.detector.
1358             auto_capture_blocks(display_duration
1359                               =2.0)
1360
1361         if captured_blocks and len(captured_blocks)
1362             > 0:
1363             print(f"\n[Camera] Detected {len(
1364                 captured_blocks)} blocks")
1365             for block in captured_blocks:
1366                 block_id = block['global_id']
1367
1368         if block_id in self.system_state.
1369             blocks:
1370             # update only XY; keep z/
1371                 stack_level/in_gripper as
1372                 they represent logical state
1373             self.system_state.blocks[
1374                 block_id]['pixel_x'] = block
1375                 ['pixel_x']
1376             self.system_state.blocks[
1377                 block_id]['pixel_y'] = block
1378                 ['pixel_y']
1379             self.system_state.blocks[
1380                 block_id]['current_position'
1381                 ]['x'] = block['pixel_x']
1382             self.system_state.blocks[
1383                 block_id]['current_position'
1384                 ]['y'] = block['pixel_y']
1385         else:
1386             # New block discovered mid-
1387                 session
1388             self.system_state.blocks[
1389                 block_id] = {
1390                 'color': block['color'],
1391                 'id': block['id'],
1392                 'pixel_x': block['pixel_x'],
1393                 'pixel_y': block['pixel_y'],
1394                 'original_x': block['pixel_x'
1395                 ],
1396                 'original_y': block['pixel_y'
1397                 ],
1398                 'current_position': {'x':
1399                     block['pixel_x'], 'y':
1400                     block['pixel_y'], 'z':
1401                     0.0},
1402                 'stack_level': 0,
1403                 'blocks_below': [],
1404                 'in_gripper': False
1405             }
1406
1407             self.system_state.rebuild_stacks()
1408             self.system_state.save_state()
1409             print("\n[Core]      Block positions
1410                 updated and saved")
1411         else:
1412             print("\n[Camera]      No blocks
1413                 detected (keeping previous positions
1414                 )")
1415
1416     def initialize(self) -> None:
1417         """Initialize all components with state
1418            loading."""
1419         print("\n" + "=" * 70)
1420         print(" ENHANCED SYSTEM INITIALIZATION v4.3"
1421             )
1422         print("=" * 70)
1423
1424         # Try to load previous state
1425         print("\n[STEP 0/3] LOADING PREVIOUS STATE")
1426         print("-" * 70)
1427         use_saved_state = False
1428         if self.system_state.load_state():
1429             print("\n Previous state loaded
1430                 successfully!")
1431             print(f" - {len(self.system_state.
1432                 blocks)} blocks remembered")
1433             print(f" - {len(self.system_state.
1434                 conversation_history)} commands in
1435                 history")
1436         try:
1437             answer = input("\nUse saved state? (
1438                 y/n): ").strip().lower()
1439         except EOFError:
1440             answer = 'y'

```

```

1403         use_saved_state = (answer == 'y')
1404
1405     # Dobot connection
1406     print("\n[STEP 1/3] DOBOT CONNECTION")
1407     print("-" * 70)
1408     if not self.dobot.connect():
1409         print("\n FATAL: Dobot connection
1410             failed")
1411         sys.exit(1)
1412
1413     # Home
1414     print("\n[STEP 2/3] POSITIONING")
1415     print("-" * 70)
1416     self.dobot.go_home()
1417     time.sleep(1)
1418
1419     if use_saved_state and self.system_state.
1420         blocks:
1421         print("\n[STEP 3/3] BLOCK DETECTION (
1422             SKIPPED - using saved state)")
1423     else:
1424         print("\n[STEP 3/3] BLOCK DETECTION")
1425         print("-" * 70)
1426         detected_blocks = self.detector.
1427             detect_all_blocks_live()
1428
1429         if not detected_blocks:
1430             print("\n FATAL: No blocks
1431                 detected")
1432             self.dobot.close()
1433             sys.exit(1)
1434
1435         self.system_state.initialize_blocks(
1436             detected_blocks)
1437
1438         print("\n" + "=" * 70)
1439         print("    INITIALIZATION COMPLETE")
1440         print("=" * 70)
1441
1442     def run_interactive_loop(self) -> None:
1443         """Main interactive command loop."""
1444         print("\n" + "=" * 70)
1445         print(" READY FOR COMMANDS - MODE v4.3")
1446         print("=" * 70)
1447         print("\nYou can give commands like:")
1448         print(" - 'pick the farthest red block and
1449             place it on the nearest green block'")
1450         print(" - 'place the blue block right next
1451             to the stack'")
1452         print(" - 'place the yellow block over the
1453             stack'")
1454         print("\nSpecial commands:")
1455         print(" - 'status' - Show current
1456             state")
1457         print(" - 'history' - Show command
1458             history")
1459         print(" - 'stacks' - Show all stacks")
1460         print(" - 'home' - Return to home")
1461         print(" - 'open camera' - Manual camera
1462             view / refresh")
1463         print(" - 'reset state' - Clear saved state
1464             on disk")
1465         print(" - 'quit' - Exit")
1466         print("=" * 70 + "\n")
1467
1468     while True:
1469         try:
1470             user_prompt = input("\n[You] Enter
1471                 command: ").strip()
1472         except EOFError:
1473             user_prompt = "quit"
1474
1475     if not user_prompt:
1476         continue

```

```

low = user_prompt.lower()

if low in ['quit', 'exit', 'q']:
    print("\n[Core] Shutting down...")
    break
elif low == 'status':
    print("\n" + "=" * 70)
    print(self.system_state.
        get_state_summary())
    print("=" * 70)
    continue
elif low == 'history':
    print("\n" + "=" * 70)
    print("COMMAND HISTORY:")
    print("-" * 70)
    for i, entry in enumerate(self.
        system_state.
        conversation_history, 1):
        print(f"{i}. '{entry['prompt']}'")
        print(f"    {entry['action']}
            at {entry['timestamp']}")
        if entry.get('details'):
            print(f"    Details: {entry['
                details']}'")
        print("=" * 70)
        continue
elif low == 'stacks':
    print("\n" + "=" * 70)
    print("CURRENT STACKS:")
    print("-" * 70)
    stacks = self.system_state.
        get_stack_locations(min_height
            =2)
    if stacks:
        for s in stacks:
            loc = s['loc_key']
            meta = self.system_state.
                stack_meta.get(loc, {})
            stack_blocks = self.
                system_state.stack_map.
                get(loc, [])
            colors = [self.system_state.
                blocks[bid]['color'] for
                bid in stack_blocks if
                bid in self.system_state.
                blocks]
            print(f"    loc {loc} @ ({s['x'
                '']},{s['y'
                '']}) : height={s
                ['height']} {
                stack_blocks} colors={
                colors}")
        else:
            print(" No stacks (height>=2)
                detected")
            print("=" * 70)
            continue
    elif low == 'home':
        self.dobot.go_home()
        continue
    elif low == 'open camera':
        self.dobot.go_home()
        time.sleep(0.5)
        captured_blocks = self.detector.
            detect_all_blocks_live()
        if captured_blocks:
            for block in captured_blocks:
                block_id = block['global_id'
                    ]
                if block_id in self.
                    system_state.blocks:
                    self.system_state.blocks
                        [block_id]['pixel_x'

```

```

        ] = block['pixel_x']
        self.system_state.blocks[
            block_id]['pixel_y']
        ] = block['pixel_y']
        self.system_state.blocks[
            block_id]['
            current_position']
            x'] = block['pixel_x']
            '']
        self.system_state.blocks[
            block_id]['
            current_position']
            y'] = block['pixel_y']
            '']
    else:
        self.system_state.blocks[
            block_id] = {
                'color': block['
                color'],
                'id': block['id'],
                'pixel_x': block['
                pixel_x'],
                'pixel_y': block['
                pixel_y'],
                'original_x': block['
                pixel_x'],
                'original_y': block['
                pixel_y'],
                'current_position':
                {'x': block['
                pixel_x'], 'y':
                block['pixel_y']
                }, 'z': 0.0},
                'stack_level': 0,
                'blocks_below': [],
                'in_gripper': False
            }
        self.system_state.rebuild_stacks()
        self.system_state.save_state()
        print(f"\n Updated {len(
            captured_blocks)} blocks")
        continue
    elif low == 'reset state':
        try:
            confirm = input("Are you sure?
                This will delete saved state
                (y/n): ").strip().lower()
        except EOFError:
            confirm = 'n'
        if confirm == 'y':
            if PERSISTENCE_FILE.exists():
                PERSISTENCE_FILE.unlink()
                print("[Core] State file
                    deleted")
            self.system_state = SystemState(
                coord_converter=self.
                system_state.coord_converter
            )
            print("[Core] State reset
                please re-detect blocks on
                next run")
            continue
        # Natural-language code via LLM
        generated_code = self.llm.generate_code(
            user_prompt, self.system_state)
        if generated_code is None:
            print("[Core] Failed to generate
                code (LLM unavailable or error)
                ")
            continue
        print("\n" + "=" * 70)
    #

print("\n" + "=" * 70)

print("GENERATED CODE:")
print("-" * 70)
print(generated_code)
print("=" * 70)

# Confirm execution
try:
    confirm = input("\nExecute this code
        ? (y/n/show): ").strip().lower()
except EOFError:
    confirm = 'n'

if confirm == 'show':
    print("\n" + generated_code)
    try:
        confirm = input("\nExecute? (y/n
            ): ").strip().lower()
    except EOFError:
        confirm = 'n'

if confirm != 'y':
    print("[Core] Execution cancelled")
    continue

success = self.executor.execute_code(
    generated_code, self.dobot, self.
    system_state)
if success:
    self.system_state.add_to_history(
        user_prompt,
        "Success",
        details={'code_length': len(
            generated_code)})
    print("\n Command completed
        successfully")
    # Optional: update from camera so
    # positions stay fresh
    self.auto_update_blocks()
else:
    self.system_state.add_to_history(
        user_prompt, "Failed")
    print("\n Command failed")
    self.dobot.go_home()

# Cleanup
print("\n[Core] Cleanup...")
try:
    self.dobot.go_home()
except Exception:
    pass
self.dobot.close()
print(f"\n System shutdown complete")
print(f" State saved to {PERSISTENCE_FILE
    }")
print("=" * 70)

def run(self) -> None:
    """Main entry point."""
    try:
        self.initialize()
        self.run_interactive_loop()
    except Exception as e:
        print(f"\n FATAL ERROR: {e}")
        import traceback
        traceback.print_exc()
    try:
        self.dobot.close()
    except Exception:
        pass

```

APPENDIX B

VISION_BLOCK_LOCATOR.PY

Computer-vision module for camera-based object detection and coordinate extraction using HSV color segmentation.

```

1617 # CLI entrypoint
1618 #
1619
1620 def parse_args() -> argparse.Namespace:
1621     parser = argparse.ArgumentParser(description="
1622         LLM-powered Dobot pick & place system")
1623     parser.add_argument(
1624         "--port",
1625         default=ROBOT_PORT,
1626         help=f"Dobot serial port (default: {
1627             ROBOT_PORT})"
1628     )
1629     parser.add_argument(
1630         "--camera-index",
1631         type=int,
1632         default=CAMERA_INDEX,
1633         help=f"Camera index (default: {CAMERA_INDEX
1634             })"
1635     )
1636     parser.add_argument(
1637         "-y", "--yes",
1638         action="store_true",
1639         help="Skip initial 'Establish CONNECTION?'
1640             confirmation"
1641     )
1642     return parser.parse_args()
1643
1644 def main() -> None:
1645     args = parse_args()
1646
1647     print(f"\n    LLM Dobot System v{__version__}"
1648         )
1649     print(f"    State File: {PERSISTENCE_FILE}")
1650     if PERSISTENCE_FILE.exists():
1651         print("    Found existing state file (will
1652             offer to load)")
1653     else:
1654         print("    No saved state (will create new)")
1655
1656     print("\n    Configuration:")
1657     print(f"    Dobot port:      {args.port}")
1658     print(f"    Camera index:    {args.camera_index}")
1659     print(f"    Frame:           {FRAME_WIDTH}x{
1660         FRAME_HEIGHT}px")
1661
1662     if args.yes:
1663         proceed = 'y'
1664     else:
1665         try:
1666             proceed = input("\n    Establish
1667                 CONNECTION? (y/n): ").strip().lower
1668             ()
1669         except EOFError:
1670             proceed = 'n'
1671
1672     if proceed == 'y':
1673         orchestrator = PickPlaceOrchestrator(port=
1674             args.port, camera_index=args.
1675             camera_index)
1676         orchestrator.run()
1677     else:
1678         print("\nExiting.")
1679
1680 if __name__ == "__main__":
1681     main()

```

```

1  #!/usr/bin/env python3
2  """
3  Vision-Based Block Detection Module
4  =====
5  Captures camera frames and identifies colored block
6  coordinates.
7
8  Features:
9  - HSV-based color detection for multiple block types
10 - Contour analysis with area filtering
11 - Returns up to 5 largest detected blocks
12 - Visual annotation support with coordinate output
13 - JSON export capability
14
15 Usage Example:
16     python3 vision_block_locator.py --show --save
17     output_frame.jpg --json block_coords.json
18
19 Configuration Notes:
20     HSV threshold values may require adjustment
21     based on lighting conditions
22     Press any key to dismiss preview window when
23     using --show flag
24 """
25
26 import argparse
27 import json
28 import sys
29 from dataclasses import dataclass
30 from typing import List, Tuple
31
32 import cv2
33 import numpy as np
34
35 # Camera Configuration
36 CAM_ID = 0
37 IMG_WIDTH = 640
38 IMG_HEIGHT = 480
39
40 # Color Detection Parameters (HSV ranges - Hue:
41 # 0-180 in OpenCV)
42 # Adjust these ranges to match your lighting
43 # environment
44 HSV_THRESHOLDS = {
45     "red": [
46         (np.array([0, 100, 90]), np.array([10, 255,
47             255])), # lower spectrum
48         (np.array([160, 100, 90]), np.array([180,
49             255, 255])), # upper spectrum
50     ],
51     "blue": [ (np.array([100, 100, 90]), np.array
52         ([130, 255, 255])) ],
53     "green": [ (np.array([40, 90, 90]), np.array
54         ([80, 255, 255])) ],
55     "yellow": [ (np.array([20, 120, 120]), np.array
56         ([35, 255, 255])) ],
57 }
58
59 # Contour filtering thresholds (pixel area)
60 AREA_MIN_THRESHOLD = 200
61 AREA_MAX_THRESHOLD = 10000
62
63 @dataclass
64 class BlockDetection:
65     """Data structure for detected block information
66     """
67     color: str

```

```

57     center: Tuple[int, int]
58     area: float
59     contour: np.ndarray
60
61
62 def identify_colored_blocks(image_frame: np.ndarray) -> List[BlockDetection]:
63     """
64     Analyzes frame for colored blocks and returns
65     detection list.
66
67     Args:
68         image_frame: BGR format image from camera
69
70     Returns:
71         List of BlockDetection objects sorted by
72         area (largest first)
73
74     """
75     hsv_image = cv2.cvtColor(image_frame, cv2.
76                             COLOR_BGR2HSV)
77     morph_kernel = np.ones((5, 5), np.uint8)
78     found_blocks: List[BlockDetection] = []
79
80     for block_color, threshold_ranges in
81         HSV_THRESHOLDS.items():
82         combined_mask = None
83         for lower_bound, upper_bound in
84             threshold_ranges:
85             single_mask = cv2.inRange(hsv_image,
86                                     lower_bound, upper_bound)
87             combined_mask = single_mask if
88             combined_mask is None else cv2.
89             bitwise_or(combined_mask,
90                     single_mask)
91
92     # Morphological operations for noise
93     # reduction
94     combined_mask = cv2.morphologyEx(
95         combined_mask, cv2.MORPH_OPEN,
96         morph_kernel)
97     combined_mask = cv2.morphologyEx(
98         combined_mask, cv2.MORPH_CLOSE,
99         morph_kernel)
100
101     detected_contours, _ = cv2.findContours(
102         combined_mask, cv2.RETR_EXTERNAL, cv2.
103         CHAIN_APPROX_SIMPLE)
104     for single_contour in detected_contours:
105         contour_area = cv2.contourArea(
106             single_contour)
107         if contour_area < AREA_MIN_THRESHOLD or
108             contour_area > AREA_MAX_THRESHOLD:
109             continue
110
111         moments = cv2.moments(single_contour)
112         if moments["m00"] == 0:
113             continue
114
115         center_x = int(moments["m10"] / moments[
116             "m00"])
117         center_y = int(moments["m01"] / moments[
118             "m00"])
119         found_blocks.append(BlockDetection(
120             color=block_color,
121             center=(center_x, center_y),
122             area=contour_area,
123             contour=single_contour
124         ))
125
126     # Sort by descending area
127     found_blocks.sort(key=lambda detection:
128                     detection.area, reverse=True)
129     return found_blocks
130
131
132 def create_annotated_visualization(image_frame: np.
133     ndarray, detected_blocks: List[BlockDetection],
134     max_blocks: int = 5):
135     """
136     Draws annotations on frame and prints detection
137     results.
138
139     Args:
140         image_frame: Original frame to annotate
141         detected_blocks: List of all detections
142         max_blocks: Maximum number of blocks to
143         process
144
145     Returns:
146         Tuple of (annotated frame, results list)
147     """
148     visualization = image_frame.copy()
149     color_map_bgr = {
150         "red": (0, 0, 255),
151         "blue": (255, 0, 0),
152         "green": (0, 255, 0),
153         "yellow": (0, 255, 255),
154     }
155
156     top_blocks = detected_blocks[:max_blocks]
157     detection_results = []
158
159     for block_num, detection in enumerate(top_blocks
160         , start=1):
161         center_x, center_y = detection.center
162         draw_color = color_map_bgr.get(detection.
163             color, (255, 255, 255))
164
165         cv2.drawContours(visualization, [detection.
166             contour], -1, draw_color, 2)
167         cv2.circle(visualization, (center_x,
168             center_y), 5, draw_color, -1)
169
170         label_text = f"{detection.color}_{block_num}"
171         ({center_x},{center_y})
172         cv2.putText(visualization, label_text, (
173             center_x - 40, center_y - 15),
174             cv2.FONT_HERSHEY_SIMPLEX, 0.5,
175             (255, 255, 255), 2)
176
177         detection_results.append({
178             "id": f"{detection.color}_{block_num}",
179             "color": detection.color,
180             "pixel_x": center_x,
181             "pixel_y": center_y,
182             "area": float(detection.area)
183         })
184
185     # Console output
186     if len(detection_results) == 0:
187         print("No blocks detected in frame.")
188     else:
189         print(f"Detected {len(detection_results)}
190             block(s):")
191         for result in detection_results:
192             print(f" - {result['id']}: ({result['
193                 pixel_x'], {result['pixel_y']}) "
194                 f"color={result['color']} area={
195                     result['area']:.1f}")
196
197     return visualization, detection_results
198
199
200 def acquire_camera_frame(camera_id: int) -> np.
201     ndarray:
202     """
203     Captures single frame from specified camera.

```

```

168 Args:
169     camera_id: Camera device index
170
171 Returns:
172     Captured frame as numpy array
173
174 Raises:
175     RuntimeError: If camera access fails
176     """
177 video_capture = cv2.VideoCapture(camera_id)
178 if not video_capture.isOpened():
179     raise RuntimeError(f"Unable to access camera
180                        at index {camera_id}")
181
182 video_capture.set(cv2.CAP_PROP_FRAME_WIDTH,
183                  IMG_WIDTH)
184 video_capture.set(cv2.CAP_PROP_FRAME_HEIGHT,
185                  IMG_HEIGHT)
186
187 # Allow camera to adjust exposure
188 for _ in range(10):
189     video_capture.read()
190
191 success, captured_frame = video_capture.read()
192 video_capture.release()
193
194 if not success or captured_frame is None:
195     raise RuntimeError("Frame capture failed")
196 return captured_frame
197
198 def main():
199     """Main execution function with argument parsing
200     """
201     argument_parser = argparse.ArgumentParser()
202     argument_parser.add_argument("--show", action="
203                                store_true",
204                                help="Display
205                                     annotated
206                                     preview window")
207     argument_parser.add_argument("--save", type=str,
208                                default=None,
209                                help="Output path
210                                     for annotated
211                                     image")
212     argument_parser.add_argument("--json", type=str,
213                                default=None,
214                                help="Output path
215                                     for JSON
216                                     coordinate data")
217     parsed_args = argument_parser.parse_args()
218
219     try:
220         camera_frame = acquire_camera_frame(CAM_ID)
221     except Exception as error:
222         print(f"ERROR: {error}")
223         sys.exit(1)
224
225     all_detections = identify_colored_blocks(
226         camera_frame)
227     if len(all_detections) < 5:
228         print(f"Warning: Expected 5 blocks but found
229               {len(all_detections)}. "
230               f"Processing available detections.")
231
232     annotated_frame, result_data =
233         create_annotated_visualization(camera_frame,
234                                       all_detections, max_blocks=5)
235
236     if parsed_args.save:
237         cv2.imwrite(parsed_args.save,
238                     annotated_frame)
239     print(f"Annotated image saved: {parsed_args.

```

```

223         save}")
224
225     if parsed_args.json:
226         with open(parsed_args.json, "w") as
227             json_file:
228                 json.dump({"blocks": result_data},
229                           json_file, indent=2)
230         print(f"Coordinate data saved: {parsed_args.
231               json}")
232
233     if parsed_args.show:
234         cv2.imshow("Block Detection Results",
235                   annotated_frame)
236         cv2.waitKey(0)
237         cv2.destroyAllWindows()
238
239 if __name__ == "__main__":
240     main()

```

APPENDIX C ROBOT_M.PY

Low-level motion control primitives and robot communica-
tion interface.

```

1 from pydobot.dobot import MODE_PTP
2 import pydobot
3 import time
4
5 # # List likely serial devices (won't error if
6 # # missing)
7 # ls -l /dev/ttyACM* /dev/ttyUSB* /dev/serial/by-id
8 # 2>/dev/null || true
9
10 device = pydobot.Dobot(port="/dev/ttyACM0")
11
12 def move_to_home(device):
13     #
14
15     # Moving the robot to a home and get home
16     # position
17     # Code
18     #####
19
20     print("Homing the robot...")
21     device.home() # Home the robot to the origin
22     # position
23     time.sleep(2)
24     (pose, joint) = device.get_pose() # Get the
25     # current position and joint angles
26     print(f"pose: {pose}, j: {joint}")
27     # Code End
28     #####
29
30     #
31
32 def move_to_specific_position(device, x, y, z, r = 0.0):
33     #
34
35     # Moving the robot to a specific position
36     # Code
37     #####
38
39     device.speed(50, 50)
40     device.move_to(mode=int(MODE_PTP.MOVJ_XYZ), x =
41                    x, y = y, z = z, r = r) # Move to position
42     # (x=250, y=0, z=50) with r=0
43     time.sleep(2)

```

<pre> 29 # Code End ##### 30 # _____ 31 32 def get_current_pose(device): 33 # _____ 34 35 # Getting the current pose 36 # Code ##### 37 38 time.sleep(1) 39 print("current pose") 40 (pose, joint) = device.get_pose() # Get the 41 # current position and joint angles 42 print(f"pose: {pose}") 43 return pose, joint 44 # Code End ##### 45 46 # _____ 47 48 # Main execution 49 50 # move_to_home(device) 51 52 # move_to_specific_position(x=327, y=62, z=-45) 53 54 get_current_pose(device) 55 # move_to_specific_position(x=218, y=-41, z=-45) 56 # move_to_specific_position(x=245, y=-53, z=-45) 57 58 # current_pose, current_joint = get_current_pose() 59 # print(f"Final pose: {current_pose}") 60 61 # Close the device connection 62 # device.close() </pre>	<pre> 17 raise RuntimeError(f"Could not open camera 18 index {cam_index}") 19 20 cap.set(cv2.CAP_PROP_FRAME_WIDTH, width) 21 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height) 22 cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) 23 24 cv2.namedWindow("Camera (q=quit, c=capture)", 25 cv2.WINDOW_NORMAL) 26 27 while True: 28 ok, frame = cap.read() 29 if not ok: 30 ok, frame = cap.read() 31 if not ok: 32 print("Frame grab failed.") 33 break 34 35 cv2.namedWindow("Camera (q=quit, c=capture)" 36 , cv2.WINDOW_NORMAL) 37 cv2.resizeWindow("Camera (q=quit, c=capture)" 38 , width, height) 39 40 h, w = frame.shape[:2] 41 put_text(frame, "Press 'c' to capture, 'q' 42 to quit", (10, 60)) 43 put_text(frame, f"Resolution: {w}x{h}", (10, 44 90)) 45 46 cv2.imshow("Camera (q=quit, c=capture)", 47 frame) 48 key = cv2.waitKey(1) & 0xFF 49 50 if key == ord('c'): 51 path = os.path.join(save_dir, f"capture. 52 png") 53 cv2.imwrite(path, frame) 54 print(f"Saved: {path}") 55 56 elif key == ord('q'): 57 break 58 59 cap.release() 60 cv2.destroyAllWindows() 61 62 if __name__ == "__main__": 63 main() </pre>
--	---

APPENDIX D OPEN_CAMERA.PY

Camera initialization utilities and frame acquisition functions.

```

1  import cv2
2  import time
3  import os
4
5  def put_text(img, text, org, scale=0.6, thickness=2)
6  :
7      cv2.putText(img, text, org, cv2.
8      FONT_HERSHEY_SIMPLEX, scale, (0, 0, 0),
9      thickness + 2, cv2.LINE_AA)
10     cv2.putText(img, text, org, cv2.
11     FONT_HERSHEY_SIMPLEX, scale, (255, 255, 255)
12     , thickness, cv2.LINE_AA)
13
14  def main():
15      cam_index = 0          # change if needed (0,
16      1, 2, ...)
17      width, height = 640 , 480
18      save_dir = "captures"
19      os.makedirs(save_dir, exist_ok=True)
20
21      cap = cv2.VideoCapture(cam_index, cv2.CAP_ANY)
22      if not cap.isOpened():

```